

UGV-Assisted Task Allocation for UAVs: A Heterogeneous Graph Reinforcement Learning Approach

Qianqian Wu¹, Qiang Liu², *Member, IEEE*, Ying He³, *Senior Member, IEEE*, and Zefan Wu¹

Abstract—Data collection and distributed task execution in Internet of Things (IoT) networks require efficient coordination among autonomous agents to handle the growing volume of sensing and computational demands. Unmanned aerial vehicles (UAVs) and unmanned ground vehicles (UGVs) present promising candidates for these operations due to their complementary capabilities and mobility advantages. However, effective cooperation between these heterogeneous agents faces significant challenges including communication limitations, energy constraints, and suboptimal task allocation efficiency. In this paper, we aim to maximize data collection capacity, task completion rates, while minimizing energy consumption across all UAVs. We propose U2GNet, a novel UGV-assisted framework for UAV that enables efficient task offloading and resource allocation in dynamic environments by leveraging Deep Reinforcement Learning (DRL) enhanced with Heterogeneous Graph Attention Networks (HGAT). The framework employs HGAT to process local observations and information shared by neighboring agents, while Gated Recurrent Units (GRU) address partial observability by integrating historical information, and DRL optimizes the decision-making process. Simulation results demonstrate that U2GNet improves the average data collection rate and task completion rate by 16.90% and 10.81% respectively compared to the baseline HGN approach.

Index Terms—Unmanned aerial vehicles (UAVs), unmanned ground vehicles (UGVs), Deep Reinforcement learning, Heterogeneous Graph Attention Network (HGAT).

I. INTRODUCTION

SMART cities leverage real-time data analytics to enable critical services such as traffic management, environmental monitoring, and public safety, with their core infrastructure relying on extensive sensor networks. With IoT-connected devices projected to reach 41.6 billion by 2025 [1], [2], data generation is increasing exponentially. Many smart city applications, including autonomous driving and disaster early warning systems, require both intensive computation and low latency. However, traditional sensors face constraints in computational capabilities

and communication bandwidth, creating a critical bottleneck in smart city development.

Unmanned Aerial Vehicles (UAVs) offer a novel solution to these challenges, leveraging their three-dimensional maneuverability, rapid deployment capabilities, and wide-area coverage advantages [3], [4], [5], [6]. Acting as intermediaries between sensor nodes and processing units, UAVs can collect data from resource-limited sensors and offload it to centralized servers, roadside units (RSUs), or other computing platforms for processing [7]. In [8], Pervez et al. designed a multi-UAV-assisted network to minimize system costs during communication. Chen et al. [9] proposed the algorithm called JTORA, which integrates Lyapunov optimization and DRL to reduce MU energy consumption in UAV-assisted MEC systems. Despite these advantages, UAVs are constrained by limited endurance and on-board computational resources [10]. For instance, frequent UAV return-to-base charging interrupts task continuity. Additionally, direct processing of massive data (such as high-definition video streams) dramatically increases local computation delays, compelling reliance on remote cloud servers and further exacerbating response latency.

To mitigate response delays from overloaded urban edge servers, a practical solution is to leverage advanced vehicle technologies for smart city development. Unmanned Ground Vehicles (UGVs), including autonomous vehicles and robots operating at the ground level, are equipped with advanced computational, communication, and storage capabilities. These features make them well-suited for performing computationally intensive tasks. Unlike fixed infrastructures such as RSUs or cloud servers, UGVs offer flexibility, which enhances both communication and computational efficiency. Wang et al. [11] proposed a vehicular fog computing system where UGVs execute computational tasks offloaded from UAVs to enhance efficiency. In [12], a UAV task offloading scheme utilizing public buses was designed to optimize UAV system costs, including energy consumption and task completion delay. In [13], Dai et al. introduced a vehicle-assisted UAV offloading architecture, leveraging mobile vehicles to enhance offloading efficiency. Thus, collaboration between UAVs and UGVs leverages the advantages of both platforms to create a heterogeneous network. This ensures that when UAV computational resources are limited, data can be efficiently offloaded to UGVs for processing.

Despite the potential benefits of UAV-UGV collaboration, several challenges must be addressed. UAVs and UGVs differ

Received 7 August 2025; revised 6 December 2025; accepted 31 December 2025. Date of publication 12 January 2026; date of current version 5 February 2026. (Corresponding author: Qiang Liu.)

Qianqian Wu, Qiang Liu, and Zefan Wu are with the Beijing Key Lab of Traffic Data Mining and Embodied Intelligence, School of Computer Science & Technology, Beijing Jiaotong University, Beijing 100044, China (e-mail: qianqianwu0921@bjtu.edu.cn; liuq@bjtu.edu.cn; zefanwu@bjtu.edu.cn).

Ying He is with the School of Electrical and Data Engineering, University of Technology Sydney, Sydney, NSW 2007, Australia (e-mail: Ying.He@uts.edu.au).

Digital Object Identifier 10.1109/TSC.2026.3651622

significantly in their mobility patterns, computational resources, and operational constraints, forming a heterogeneous system. Conventional methods that rely on fixed or homogeneous network representations fail to capture the diversity and dynamics of interactions between heterogeneous agents. For instance, when a UAV has limited computing resources, computational tasks need to be offloaded to UGVs, requiring careful consideration of their positions, available computational resources, and current workloads. Consequently, the key challenge lies in designing an advanced UAV-UGV collaboration strategy that can adapt to dynamic environments while ensuring efficient system operation.

To address the issue, We propose U2GNet, an innovative UAV-UGV framework that integrates Heterogeneous Graph Attention Network (HGAT) with Deep Reinforcement Learning (DRL). U2GNet constructs a communication mechanism among heterogeneous agents and processes both local observations and neighbor-transmitted messages of UAVs and UGVs through HGAT. We incorporate Gated Recurrent Unit (GRU) to store historical information, thereby addressing partial observability issues. Finally, we employ DRL for policy optimization to maximize long-term rewards, improving data collection and task completion rates while reducing UAV energy consumption.

The main contributions of this paper are as follows:

- We introduce U2GNet, a novel UAV-UGV framework leveraging UAV mobility and UGV computational capabilities to enhance collaboration efficiency in heterogeneous environments.
- We propose a DRL method enhanced by a HGAT, which enables information exchange between UAVs and UGVs through HGAT-based message aggregation. In addition, a GRU-based memory module is integrated to incorporate historical information into the decision-making process.
- Extensive simulations validate the effectiveness of U2GNet, demonstrating its superior performance over baseline methods in data collection, task completion, and energy consumption.

The remainder of the paper consists of the following sections. Section II discusses related work and analyses the contributions and shortcomings of existing work. Section III presents the system model and problem formulation of the paper. We describe our approach in Section IV. Section V gives simulation results to evaluate the performance of the proposed approach through simulation experiments, and concludes the paper in Section VI.

II. RELATED WORK

Due to the limited computational capabilities of the devices, UAVs struggle to independently process computationally intensive tasks [14]. Several existing works focus on offloading UAV tasks to edge servers to address their energy constraints. In [15], Tang et al. proposed a multi-UAV-assisted MEC framework based on short-task priority queuing, optimizing user device latency and UAV load fairness. Chen et al. [16] proposed a joint UAV and low earth orbit satellite task offloading algorithm to achieve nash equilibrium offloading strategies. To enable UAVs to respond in real-time to meet dynamic user preferences in multi-access edge computing, Huang et al. [17] developed a

multi-objective DRL approach for UAV computing offloading and trajectory control. In [18], Lyu et al. proposed a multi-agent DRL (MADRL) computation offloading algorithm that reduces total system costs. Nidhi et al. [19] studied the joint optimization of uplink and downlink resource allocation and task offloading based on an improved double deep Q-network scheme, minimizing task latency. Wang et al. [20] proposed a multi-UAV MEC system with RIS, optimizing UAV trajectories, offloading, and RIS phase shifts using DRL to reduce delay and improve fairness. In [21], the dependencies between computational tasks were considered, and a task offloading algorithm based on asynchronous federated DRL was proposed to enhance data diversity while ensuring data privacy.

However, the fixed deployment pattern of edge servers creates a spatial mismatch with the high mobility of UAVs, giving rise to a new paradigm of cross-platform collaborative offloading. An increasing number of studies have begun to consider offloading UAV tasks to ground mobile devices (such as UGVs). In [22], Xu et al. proposed an innovative two-layer distributed online task scheduling method that optimizes UGV resource utilization, reducing the task latency and the energy consumption. Chen et al. [23] maximized the average efficiency of UAV-UGV collaboration using an iterative algorithm based on block coordinate descent and continuous convex approximation techniques. In [24], Yan et al. proposed a software-defined network (SDN)-driven UAV-assisted VEC network partial offloading model. gMADRL-VCS [25] optimizes the UGV and UAV data collection collaboration in post-disaster rescue operations, demonstrating excellent energy efficiency and collaborative performance in real datasets. Zhang et al. [26] proposed a novel edge offloading system for UAV and mobile edge service vehicle collaboration based on an improved dung beetle optimizer (DBO) algorithm and twin-delayed deep deterministic policy gradient (TD3) algorithm. In [27], Qin et al. developed a multi-layer collaborative IoV model based on MADRL, which addresses adversarial competition and resource scheduling among vehicles through the coordinated deployment of UAVs, RSUs, and parked cars serving as RSUs (PCRSUs).

However, the aforementioned studies model UAVs and UGVs as independent computing entities, with limited exploration of collaboration mechanisms between these heterogeneous nodes, resulting in inefficient cross-platform collaboration. In contrast to previous strategies, we propose a collaborative framework based on DRL enhanced by HGAT. The framework captures complex spatiotemporal dependencies among heterogeneous devices through HGAT, while employing DRL to optimize decision-making in dynamic scenarios. This approach offers a novel paradigm for enhancing the performance of UAV-UGV collaborative systems.

III. SYSTEM MODEL AND PROBLEM FORMULATION

We have constructed a data collection and offloading scenario based on UAV-UGV collaboration, as depicted in Fig. 1. Within the target region $\mathcal{Q}$, a set of Points of Interest (PoIs) are randomly distributed. Time is discretized into time slots, such that the total task duration of the UAV consists of T time slots, each with a

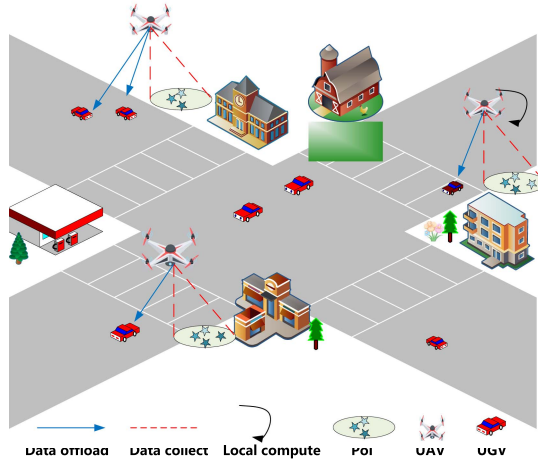


Fig. 1. UAV-UGV framework to perform data collection and task allocation scenarios.

duration of τ seconds. The sets of UAVs and UGVs are defined as $\mathcal{U} = \{u_1, u_2, \dots, u_U\}$ and $\mathcal{V} = \{v_1, v_2, \dots, v_V\}$, respectively. UAVs fly at a fixed altitude h_u , with their positions at time t represented as $q_u^{(t)} = (x_u^{(t)}, y_u^{(t)}, h_u)$. The positions of UGVs are represented as $q_v^{(t)} = (x_v^{(t)}, y_v^{(t)})$. The motion of the UAV can be described as:

$$x_u^{(t+1)} = x_u^{(t)} + v_u^{(t)} \cos \theta_u^{(t)} \tau \quad (1a)$$

$$y_u^{(t+1)} = y_u^{(t)} + v_u^{(t)} \sin \theta_u^{(t)} \tau \quad (1b)$$

Where $v_u^{(t)}$ and $\theta_u^{(t)}$ denote the speed and heading angle of the UAV at the slot t . The UGV's position is updated as:

$$x_v^{(t+1)} = x_v^{(t)} + v_v^{(t)} \cdot \cos(\theta_v^{(t)}) \cdot \tau \quad (2a)$$

$$y_v^{(t+1)} = y_v^{(t)} + v_v^{(t)} \cdot \sin(\theta_v^{(t)}) \cdot \tau \quad (2b)$$

where $\theta_v^{(t)}$ is the moving direction angle of UGV v at time slot t .

A. Communication Model

We assume that UAVs maintain hovering during PoI data collection and offloading, eliminating the Doppler effect to simplify model complexity. Let $d_{u,v}^{(t)}$ represent the distance between UAV u and UGV v , given by the following equation:

$$d_{u,v}^{(t)} = \sqrt{(x_u^{(t)} - x_v^{(t)})^2 + (y_u^{(t)} - y_v^{(t)})^2 + (h_u^{(t)})^2} \quad (3)$$

The air-to-ground channel is typically categorized into two scenarios: Line-of-Sight (LoS) and Non-Line-of-Sight (NLoS), both of which include free space path loss and additional path loss [28]. The path loss function between UAV nodes and UGV nodes can be defined as follows:

$$L_{u,v} = \zeta_{LoS} \Lambda_{LoS} + \zeta_{NLoS} \Lambda_{NLoS} \quad (4)$$

where ζ_{LoS} and ζ_{NLoS} represent the probabilities of LoS and NLoS propagation, respectively, while Λ_{LoS} and Λ_{NLoS} represent the path loss under LoS and NLoS conditions, respectively.

We can get ζ_{LoS} and ζ_{NLoS} :

$$\begin{cases} \zeta_{LoS} = \frac{1}{1+a \exp(-b[\varpi-a])} \\ \zeta_{NLoS} = 1 - \zeta_{LoS} \end{cases} \quad (5)$$

where ϖ is the elevation angle between the UAV and the UGV, a and b are constant values depending on the environment (e.g., city, suburbs, rural areas, etc.).

The path loss can be expressed as:

$$\Lambda_\ell = \begin{cases} \eta_L + 20 \log_{10} (4\pi f_c d_{u,v} \mu^{-1}), & \ell = LoS \\ \eta_N + 20 \log_{10} (4\pi f_c d_{u,v} \mu^{-1}), & \ell = NLoS \end{cases} \quad (6)$$

where η_L and η_N are the additional path losses associated with the environment, f_c is the carrier frequency, and μ is the speed of light. Substituting Λ_ℓ into equation (4), we obtain:

$$L_{u,v} = \zeta_{LoS} \eta_L + \zeta_{NLoS} \eta_N + 20 \log_{10} (4\pi f_c d_{u,v} \mu^{-1}) \quad (7)$$

The channel gain is calculated from the linear value of the path loss:

$$\begin{cases} L_{u,v}^{linear} = 10^{\frac{L_{u,v}}{10}} \\ g_{u,v} = \frac{1}{L_{u,v}^{linear}} = 10^{-L_{u,v}/10} \end{cases} \quad (8)$$

When UAV u transmits data to UGV v , signals emitted by other UAVs introduce co-channel interference. To mitigate this interference, we adopt a non-orthogonal multiple access (NOMA) scheme at the UGV receiver, combined with successive interference cancellation (SIC) [29]. We define a neighborhood observation radius R_{obs} . When $d_{u,v}^{(t)} \leq R_{obs}$, UAV u and UGV v are considered to establish an effective communication link; otherwise, the link is regarded as unavailable. Accordingly, in each time slot t , the set of UAVs that are connected to UGV v is given by $\mathcal{U}_v^{(t)} = \{u : d_{u,v}^{(t)} \leq R_{obs}\}$.

To characterize the SIC decoding order, the UAVs in $\mathcal{U}_v^{(t)}$ are sorted in ascending order of their channel gains $g_{u,v}$ (i.e., the weakest channel is decoded first). Let $u_{(i)}$ denote the i -th UAV in this ordered list. When decoding the signal of UAV $u_{(i)}$, residual interference from all stronger UAVs $u_{(j)}$ with $j > i$ remains. For the i -th UAV $u_{(i)} \in \mathcal{U}_v^{(t)}$, its SINR at UGV v during decoding is given by:

$$\text{SINR}_{u_{(i)},v}^{(t)} = \frac{g_{u_{(i)},v} P_{u_{(i)}}^{\text{tx}}}{\sigma^2 + \sum_{j>i} g_{u_{(j)},v} P_{u_{(j)}}^{\text{tx}}}, \quad (9)$$

where σ^2 denotes the noise power, and P_u^{tx} is the time-varying transmit power of UAV u , which satisfies $0 < P_u^{\min} \leq P_u^{\text{tx}}(t) \leq P_u^{\max}$, $\forall u, t$.

Consequently, the instantaneous transmission rate from UAV $u_{(i)}$ to UGV v is expressed as:

$$R_{u_{(i)},v}^{(t)} = b_{u,v} \log_2 \left(1 + \text{SINR}_{u_{(i)},v}^{(t)} \right), \quad (10)$$

where $b_{u,v}$ denotes the bandwidth of the wireless channel.

B. Offload Model

The PoIs are represented by the set $\mathcal{M} = \{m_1, m_2, \dots, m_M\}$. Let $D_m^{(0)}$ denote the data size of PoI m . The entire task execution

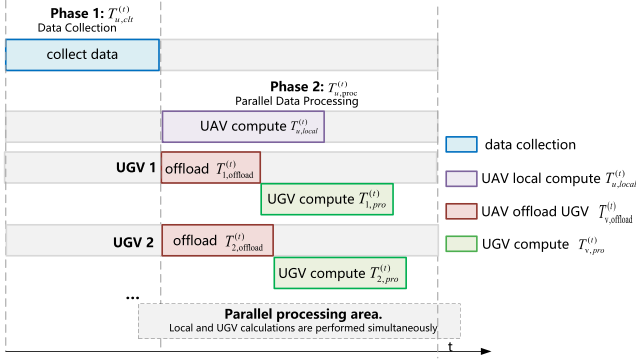


Fig. 2. Illustration of the two-phase task execution process of a UAV in collaboration with UGVs. Phase 1 is the UAV data acquisition process, and in Phase 2, the UAV processes the data partially locally and partially offloads it to the UGV for computation.

process of a UAV in one time slot can be divided into two sequential phases: data collection and processing (local computing or offloading). Fig. 2 illustrates the time allocation and interactions among UAVs and UGVs across the two phases.

Each UAV u collects data during a predetermined time period $T_{u,clt}^{(t)}$. We assume that UAVs employ a round-robin policy to collect data from all covered PoIs within their sensing range (defined as subset $M' \subset \mathcal{M}$). This round-robin policy divides the time period $T_{u,clt}^{(t)}$ uniformly to accommodate the number of covered PoIs, which is a common approach in control systems [5], [30]. Based on this, we define $D_{u,m}^{(t)}$ the data collected by UAV u from PoI m , as follows:

$$D_{u,m}^{(t)} = \min \left(v_c \frac{T_{u,clt}^{(t)}}{|M'|}, D_m^{(t)} \right) \quad (11)$$

where $D_m^{(t)}$ denotes the total remaining data of PoI m covered by the UAV in time slot t , and v_c represents the sensor sampling frequency of the UAV. Then, the total amount of data collected by UAV u during time slot t is given by:

$$D_u^{(t)} = \sum_{m \in M'} D_{u,m}^{(t)}. \quad (12)$$

The energy consumption for the data collection process is given by:

$$e_{u,clt}^{(t)} = P_u^c T_{u,clt}^{(t)}, \quad (13)$$

where P_u^c denotes the power consumption of UAV u during data collection. At the end of each time slot, the remaining amount of data for each covered PoI is updated as follows:

$$D_m^{(t+1)} = \max \left(D_m^{(t)} - D_{u,m}^{(t)}, 0 \right), m \in M'. \quad (14)$$

When a UAV lacks sufficient computational resources to process the collected data, it needs to offload the data to nearby UGVs. We assume parallel transmissions among multiple UGVs and dynamic allocation of computational resources between UAVs and UGVs using Dynamic Voltage and Frequency Scaling (DVFS) [12], [31], thereby enabling partial data offloading.

The UAV looks for UGVs within R_{obs} for data offloading, and we can get the set of UGVs for data offloading as:

$$\mathcal{N}_u^{(t)} = \left\{ v \mid d_{u,v}^{(t)} \leq R_{obs} \right\} \quad (15)$$

The time required for UAV u to compute data locally can be expressed as:

$$T_{u,local}^{(t)} = \varsigma_u \frac{D_u^{(t)} \cdot C_{u,m}}{f_u^{(t)}}, \quad (16)$$

where $C_{u,m}$ denotes the number of CPU cycles required for UAV u to compute each bit of data, and $f_u^{(t)}$ is the available computational capability of UAV u during time slot t , constrained by $f_u^{(t)} \leq f_{u,max} \cdot \varsigma_u$ and ς_v represent the proportion of data processed on UAV u and UGV v respectively, satisfying:

$$\varsigma_u + \sum_{v \in \mathcal{N}_u^{(t)}} \varsigma_v = 1, \quad 0 \leq \varsigma_u, \varsigma_v \leq 1, \quad (17)$$

when $\mathcal{N}_u^{(t)} = \emptyset$.

The energy consumption of UAV u for local computing is given by:

$$e_{u,local}^{(t)} = \kappa_u \cdot \left(f_u^{(t)} \right)^3 \cdot T_{u,local}^{(t)}, \quad (18)$$

where κ_u is the capacitance coefficient of UAV u .

The transmission time required for UAV u to offload a portion of its data to UGV v is expressed as:

$$T_{v,offload}^{(t)} = \varsigma_v \frac{D_u^{(t)}}{R_{u,v}^{(t)}}. \quad (19)$$

The energy consumption during the data transmission process is calculated as:

$$e_{u,offload}^{(t)} = P_u^{tx} T_{u,offload}^{(t)}. \quad (20)$$

After receiving the data, UGV v processes it with a corresponding processing delay given by:

$$T_{v,proc}^{(t)} = \varsigma_v \frac{D_u^{(t)} C_{v,m}}{f_v^{(t)}}, \quad (21)$$

where $C_{v,m}$ denotes the number of CPU cycles required for UGV v to compute each bit of data, and $f_v^{(t)}$ is the available computational capability of UGV v during time slot t , constrained by $f_v^{(t)} \leq f_{v,max}$.

The total task completion time of UAV u during time slot t , denoted as $T_{u,total}^{(t)}$, is expressed as:

$$\begin{cases} T_{u,proc}^{(t)} = \max \left(T_{u,local}^{(t)}, \max_{v \in \mathcal{N}_u^{(t)}} \left(T_{v,offload}^{(t)} + T_{v,proc}^{(t)} \right) \right) \\ T_{u,total}^{(t)} = T_{u,clt}^{(t)} + T_{u,proc}^{(t)} \end{cases} \quad (22)$$

To ensure that all operations are completed within a single time slot, the total time spent by UAV u during time slot t is constrained by an upper bound τ_{max} , such that: $T_{u,total}^{(t)} \leq \tau_{max}$. We assume that the time slot is sufficiently large and set $\tau_{max} = \tau$.

The total energy consumption of UAV u in time slot t is given by:

$$e_{u,total}^{(t)} = \sum_{v \in \mathcal{N}_u^{(t)}} e_{u,offload}^{(t)} + e_{u,local}^{(t)} + e_{u,clt}^{(t)} + e_{u,move}^{(t)}. \quad (23)$$

This total energy consumption satisfies $e_{u,total}^{(t)} \leq E_{u,0}$, where $E_{u,0}$ denotes the initial energy reserve of each UAV. The propulsion energy consumption is expressed as:

$$e_{u,move}^{(t)} = \frac{1}{2} \varphi_d \rho A_f (V_v + V_{wind})^3 \tau, \quad (24)$$

where φ_d is the drag coefficient, ρ is the air density, A_f is the frontal area of the UAV, V_v is the UAV's flight speed, and V_{wind} is the ambient wind speed, which is modeled as a random variable in this work [32].

C. Problem Formulation

In this section, we formulate an energy-efficient optimization problem for UAV-UGV collaboration, aiming to enable effective data collection and computing by UAVs. Upon completion of the data collection task, the data collection ratio is calculated as:

$$\eta^{(t)} = \frac{\sum_{u=1}^U \sum_{m=1}^M D_{u,m}^{(t)}}{\sum_{m=1}^M D_m^{(0)}}. \quad (25)$$

The task completion rate is the number of tasks completed within τ_{max} divided by the total number of tasks, defined as:

$$\xi^{(t)} = \frac{\sum_{u=1}^U \left[\left(\sum_{v \in \mathcal{N}_u^{(t)}} \varsigma_v^{(t)} D_u^{(t)} \right) + \varsigma_u^{(t)} D_u^{(t)} \right]}{\sum_{u=1}^U D_u^{(t)}}. \quad (26)$$

The UAV energy consumption index is defined as the ratio of the total energy consumed by all UAVs over time to their total initial energy reserves:

$$\psi^{(t)} = \frac{\sum_{t=1}^T \sum_{u=1}^U e_{u,total}^{(t)}}{T \cdot U \cdot E_{u,0}}, \quad (27)$$

where $E_{u,0}$ denotes the initial energy reserve of each UAV.

We formulate the problem of optimizing offloading ratios for UAVs and UGVs to maximize computational efficiency. Let $\ell^{(t)}$ denote the overall computation efficiency at time slot t , including the data collection rate, task completion rate, energy consumption, and task completion time. $\ell^{(t)}$ is defined as:

$$\ell^{(t)} = \omega_1 \eta^{(t)} + \omega_2 \xi^{(t)} - \omega_3 \psi^{(t)} - \omega_4 \chi^{(t)} \quad (28)$$

where $\chi^{(t)} = \frac{1}{U} \sum_{u=1}^U \frac{T_{u,total}}{\tau_{max}}$. $\omega_1, \omega_2, \omega_3, \omega_4$ are weight coefficients used to balance the importance of different objectives and $\sum_{i=1}^4 \omega_i = 1$. According to Eq. (28), the optimization problem can be formulated as:

$$\mathbf{P1}: \max_{\varsigma_u, \varsigma_v, f_u^{(t)}, f_v^{(t)}} \ell^{(t)} \quad (29)$$

$$\text{s.t. C1: } \varsigma_u + \sum_{v \in \mathcal{N}_u^{(t)}} \varsigma_v \leq 1, \quad \forall u, t \quad (29a)$$

$$\text{C2: } 0 \leq \varsigma_u^{(t)}, \varsigma_v^{(t)} \leq 1, \quad \forall u, v, t \quad (29b)$$

$$\text{C3: } f_u^{(t)} \leq f_{u,max}, \quad \forall u, t \quad (29c)$$

$$\text{C4: } f_v^{(t)} \leq f_{v,max}, \quad \forall v, t \quad (29d)$$

Constraints C1 and C2 ensure the validity of data processing ratios; C3 and C4 limit the usage of computational resources.

IV. ALGORITHM DESIGN

Given the nonconvex nature of problem **P1**, we reformulate it as a MDP. The MDP is defined as a tuple $\mathcal{MDP} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, where $\mathcal{S}$ represents the state space, $\mathcal{A}$ denotes the action space, $\mathcal{P}$ is the state transition probability function, and $\mathcal{R}$ is the reward function.

- *state*: The state space $\mathcal{S}$ at time slot t is defined as: $s^{(t)} = \{q_u^{(t)}, q_v^{(t)}, E_u^{(t)}, D_m^{(t)}, \mathcal{N}_u^{(t)}\}$, where $q_u^{(t)}$ and $q_v^{(t)}$ represent the positions of UAVs and UGVs respectively, $D_m^{(t)}$ denotes the remaining data at PoI m , $\mathcal{N}_u^{(t)}$ is the set of UGVs available for offloading, and $E_u^{(t)} = E_{u,0} - \sum_t e_{u,total}$ denotes the remaining energy.
- *action*: The action space $\mathcal{A}$ at time slot t consists of the computational resource allocation and data offloading decisions: $a^{(t)} = \{\varsigma_u^{(t)}, \varsigma_v^{(t)}, f_u^{(t)}, f_v^{(t)}, \forall u \in \mathcal{U}, \forall v \in \mathcal{N}_u^{(t)}\}$, where $\varsigma_u^{(t)}$ and $\varsigma_v^{(t)}$ represent the proportion of data processed locally by UAV u and offloaded to UGV v respectively, $\sum_{v \in \mathcal{N}_u^{(t)}} \varsigma_v^{(t)} + \varsigma_u^{(t)} = 1$, while $f_u^{(t)}$ and $f_v^{(t)}$ denote the computational capabilities allocated to UAV u and UGV v .
- *reward*: Finally, a reward r_t is obtained at the end of each time slot to evaluate the effectiveness of the corresponding decision. The reward is computed as:

$$r_t = \underbrace{\omega_1 \eta^{(t)} + \omega_2 \xi^{(t)}}_{\text{positive}} - \underbrace{\omega_3 \psi^{(t)} + \omega_4 \chi^{(t)}}_{\text{negative}}, \quad (30)$$

where the weights ω_1 to ω_4 are normalized such that $\sum_{i=1}^4 \omega_i = 1$.

A. Feature Extraction

The entities in the environment (UAVs and UGVs) are categorized into L groups, represented by the set $\mathcal{L} = \{1, 2, \dots, L\}$, where each group corresponds to entities with similar characteristics. For any agent $i \in \mathcal{I}$, its observation range $\mathcal{O}_i$ and communication range $\mathcal{C}_i$ are defined as the sets of all nodes within its observation radius R_{obs} and communication radius R_{comm} , respectively, i.e.:

$$\mathcal{O}_i = \{j \mid j \in \mathcal{I}, d_{ij} \leq R_{obs}\}, \quad (31a)$$

$$\mathcal{C}_i = \{j \mid j \in \mathcal{I}, d_{ij} \leq R_{comm}\}, \quad (31b)$$

where $d_{ij} = \|\mathbf{q}_i - \mathbf{q}_j\|$ denotes the Euclidean distance between agent i and j .

As shown in Fig. 3, the observation graph G_o^i and the communication graph G_c^i of agent i are constructed based on $\mathcal{O}_i$ and $\mathcal{C}_i$, respectively. The adjacency matrix of G_o^i , denoted as A_o^i , is defined as follows:

$$A_o^i[j, u] = \begin{cases} 1 & \text{if } j \in \mathcal{O}_i, u \in \mathcal{O}_i, d_{ju} \leq R_{obs} \\ 0 & \text{otherwise} \end{cases}, \quad (32)$$

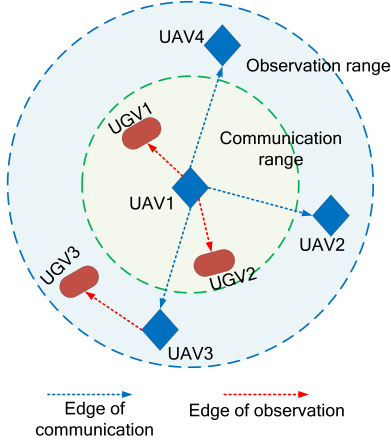


Fig. 3. Observation and communication ranges in multi-agent systems. Blue circles indicate the observation range, where entities can be observed, and green circles represent the communication range for direct communication. Red dashed lines show relationships within the observation range, while blue dashed lines depict communication links.

where $A_o^i[j, u] = 1$ indicates that agent j and agent u can be observed by agent i . Otherwise, $A_o^i[j, u] = 0$.

Similarly, the adjacency matrix of the communication graph G_c^i is defined as A_c^i :

$$A_c^i[j, u] = \begin{cases} 1, & \text{if } j, u \in C_i \text{ and } d_{ju} \leq R_{comm}, \\ 0, & \text{otherwise} \end{cases}, \quad (33)$$

where $A_c^i[j, u] = 1$ indicates that agent i can directly communicate with agents j and u . Otherwise, $A_c^i[j, u] = 0$.

For each agent i belonging to group $l \in \mathcal{L}$, its raw state s_i^l is processed by a group-specific encoder to produce a high-dimensional embedding:

$$\mathbf{e}_i^l = f_e^l(s_i^l; \Theta_e^l), \quad l \in \mathcal{L}, \quad (34)$$

where f_e^l denotes the encoder function parameterized by Θ_e^l . Different entity types employ distinct encoders to capture their unique characteristics.

1) *Intra-Group Observation Aggregation*: In the first HGAT layer, for agent i in group l and each neighbor j within its observation range, the attention weight α_{ij}^l quantifies the importance of neighbor j to agent i . For each attention head h , this weight is computed as:

$$\alpha_{ij}^{l,(h)} = \frac{\exp\left(\frac{(\mathbf{W}_q^{(h)} \mathbf{e}_i)^T (\mathbf{W}_k^{(h)} \mathbf{e}_j)}{\sqrt{d_{(h)}}}\right)}{\sum_{u \in \mathcal{O}_i} \exp\left(\frac{(\mathbf{W}_q^{(h)} \mathbf{e}_i)^T (\mathbf{W}_k^{(h)} \mathbf{e}_u)}{\sqrt{d_{(h)}}}\right)}, \quad (35)$$

where $\mathbf{W}_q^{(h)}$ and $\mathbf{W}_k^{(h)}$ are learnable matrices that project embeddings into queries and keys, respectively, and $d_{(h)}$ is the dimensionality of head h .

After computing the attention weights, agent i aggregates information from its observable neighbors j to obtain the local embedding $\mathbf{e}_{i,(h)}^l$:

$$\mathbf{e}_{i,(h)}^l = \sum_{j \in \mathcal{O}_i} \alpha_{ij}^{l,(h)} \mathbf{W}_v^l \mathbf{e}_j, \quad (36)$$

where $\mathbf{W}_v^l$ is a type-specific value projection matrix for nodes of group l . Unlike homogeneous GATs, which share a single value projection across all nodes, our approach employs independent $\mathbf{W}_v^l$ for different entity types. By concatenating the outputs from all heads, we can get:

$$\mathbf{e}_i^l = \text{Concat}(\mathbf{e}_{i,1}^l, \mathbf{e}_{i,2}^l, \dots, \mathbf{e}_{i,(h)}^l). \quad (37)$$

2) *Inter-Group Aggregation*: Once the intra-group information aggregation is completed, we need to aggregate the information from different groups to obtain agent i 's global perception of the environment.

HGAT integrates the results from different groups through a fully connected layer f_G . We obtain a unified representation $\mathbf{e}_i^l$ that includes cross-group information:

$$\mathbf{e}_i^l = f_G\left(\|_{l \in \mathcal{O}_i^L} \mathbf{e}_i^l\right), \quad (38)$$

where $\|$ denotes the concatenation operation, and $\mathcal{O}_i^L$ represents the set of agents selected by agent i based on A_O .

3) *Inter-Agent Communication*: The second layer implements explicit communication between agents. Agent j transmits its representation $\mathbf{e}_j^l$ as the message $\text{Msg}(ji)$ to agent i within its communication range, enabling information sharing with neighbors. The message is defined as:

$$\text{Msg}(ji) = \begin{cases} \mathbf{e}_j^l, & \text{if } j \in C_i \\ 0, & \text{otherwise} \end{cases}, \quad (39)$$

where $\text{Msg}(ji)$ represents the aggregated environmental information from agent j shared with its neighbors.

After agent i receives $\text{Msg}(ji)$, similar to the first layer, the second layer of HGAT processes $\text{Msg}(ji)$ based on A_c and aggregates it into another embedding vector $\mathbf{e}_i''$.

4) *Input Concatenation*: We concatenate the output vectors $\mathbf{e}_i^l$ and $\mathbf{e}_i''$ from the second layer of HGAT to obtain the new embedding $\mathbf{e}_i$ at time slot t . Considering that agents need to capture critical historical information from the sequential data, we introduce a GRU to process the node embeddings, thereby retaining temporal dependencies in the hidden state:

$$h_i^t = \text{GRU}(\mathbf{e}_i^t, h_i^{t-1}), \quad (40)$$

where h_i^{t-1} represents the hidden state from the previous time slot, which encodes the important features of historical node information.

Subsequently, the GRU output is transformed into a state representation $s_t = \text{MLP}(h_i^t)$ through full connectivity layer. We have summarized the feature extraction process in Fig. 4.

B. DRL Decision

The action selection of agent i follows the policy $a_i^t \sim \pi_{\theta_i}(a_i^t | s_i^t)$. Following the backbone of IPPO [33], we employ one policy network π_{θ} and one value network V_{θ} , with the function defined as:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\min \left(\tau_t(\theta) \hat{A}_t, \text{clip}(\tau_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (41)$$

where $\tau_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ denotes the probability ratio between the updated and the previous policy, and $\hat{A}_t$ is the estimated

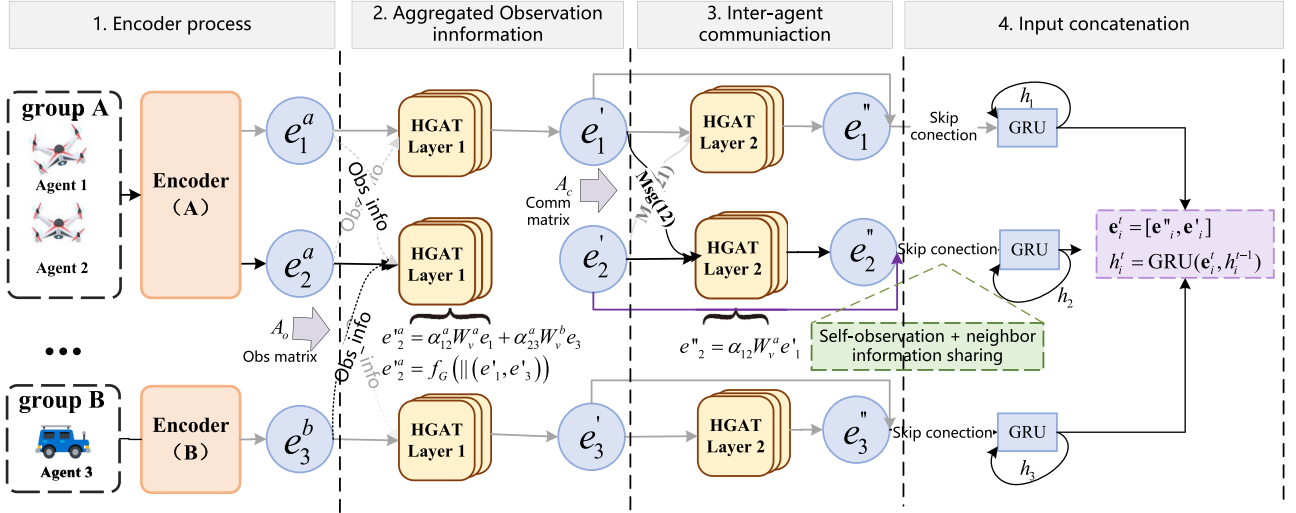


Fig. 4. Diagram of the feature extraction process. For simplicity, the figure depicts three agents from two groups: agents 1 and 2 belong to group A, while agent 3 belong to group B. Each agent's state is first encoded using a group-specific encoder (Encoder A for group A, Encoder B for group B). agent 2 is interconnected with agent 1 and agent 3.

value of the advantage function. ϵ is a hyperparameter used to control the magnitude of policy updates, typically set as $\epsilon = 0.2$.

We adopt the GAE to compute the advantage value $\hat{A}_t$, defined as:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad (42)$$

where $\delta_t = r_t + \gamma V_\vartheta(s_{t+1}) - V_\vartheta(s_t)$ is the TD residual, γ is the discount factor, and λ is the parameter for GAE.

The value network V_ϑ are updated by minimizing the following loss function:

$$\mathcal{L}(\vartheta) = \mathbb{E} \left[\max \left((V_\vartheta(s_t) - \hat{R}_t)^2, \right. \right. \\ \left. \left. (\text{clip}(V_\vartheta(s_t), V_{\vartheta_{old}}(s_t) - \epsilon_1, V_{\vartheta_{old}}(s_t) + \epsilon_1) - \hat{R}_t)^2 \right) \right], \quad (43)$$

where $V_\vartheta(s_t)$ is the value function for state s_t , and $\hat{R}_t$ is the estimated cumulative return.

To encourage policy exploration, an entropy regularization term is incorporated:

$$\mathcal{L}^S(\theta) = -\mathbb{E}_{a \sim \pi_\theta} [\log \pi_\theta(a_t | s_t)], \quad (44)$$

The total loss function is expressed as:

$$\mathcal{L}(\theta, \vartheta) = \mathcal{L}(\theta) + c_1 \mathcal{L}(\vartheta) + c_2 \mathcal{L}^S(\theta), \quad (45)$$

where c_1 and c_2 are hyperparameters that balance the contributions of the value function loss and entropy regularization. Pseudo-codes of U2GNet are shown in Algorithm 1.

C. Complexity Analysis

In a scenario involving L groups with M entities (including N agents), for each agent i with M_0 entities within its observation range, the computational complexity of feature extraction is

TABLE I
EXPERIMENTS PARAMETERS OF SETTINGS [12], [34]

Parameters	Values (units)	Parameters	Values (units)
η_L, η_N	1, 23	a, b	9.61, 0.16
C_m	[50,150]	σ^2	-104 (dBm)
P_c	5 (W)	κ_u	10^{-28}
$b_{u,v}$	10 (MHz)	P_u^{tx}	33 (dBm)
$E_{u,0}$	2 KJ	f_u	[5,9] GHz
f_v	[5,15] GHz	$\tau_{\max}$	800 (ms)
R_{comm}	150 (m)	R_{obs}	90 (m)
$\omega_1, \omega_2, \omega_3, \omega_4$	0.25	v_m	30 (m/s)

$O(M_o \cdot K \cdot d_s)$. The most complex case is when $M_0 = M$, resulting in a complexity of $O(M \cdot K \cdot d_s)$, where K is the number of attention heads and d_s is the feature dimension. The inter-group feature aggregation has a complexity of $O(L \cdot d_s)$. The second layer of HGAT performs computations based on the communication range C_i , with a computational complexity of $O(N_c \cdot K \cdot d_s)$, where N_c represents the number of entities within agent i 's communication range. The computational complexity of the GRU unit is $O(d_s^2)$. The computational complexity of the decision process primarily comes from the forward and backward propagation of the policy network and value network, which is $O(d^2 + T)$. In summary, the total complexity is $O(M_o \cdot K \cdot d_s + L \cdot d_s + N_c \cdot K \cdot d_s + d_s^2 + d^2 + T)$. When $N_c \approx M$, the overall computational complexity for each agent i is approximately $O(M)$.

V. NUMERICAL RESULTS AND DISCUSSIONS

In this section, we evaluate the proposed algorithm and compare it with several state-of-the-art methods.

Algorithm 1: Pseudocode of U2GNet.

```

Initialize parameters of  $\theta, \vartheta, s$ ;
for  $Episode$  in  $= 1, 2, \dots$  do
    Reset environment, initialize states  $s^{(0)}$ ;
    for each time step  $t$  do
        for agent  $i \in \mathcal{I}$  do
            Compute  $\mathcal{O}_i, C_i$ ;
            Get the embedding vector by Eq.(34);
            for each attention head  $k=1, \dots, K$  do
                compute attention weights by Eq.(35);
                Aggregate neighbor feature by
                 $\mathbf{e}_{i,k}^l = \sum_{j \in \mathcal{O}_i} \alpha_{ij}^k \mathbf{W}_v^k \mathbf{e}_j^l$ ;
            end
            Concatenate all attention head outputs by
            Eq.(37);
             $\mathbf{e}_i' = f_G(\|\|_{l \in \mathcal{O}_i^L} \mathbf{e}_i^l)$ ;
            Receive  $Msg$  by Eq.(39);
            Repeating the first layer's process, to obtain
             $\mathbf{e}_i''$ ;
             $h_i' = \text{GRU}([\mathbf{e}_i', \mathbf{e}_i''], h_i)$ ;
             $s_t = \text{MLP}(h_i')$ ;
            Sample actions according to  $\pi_\theta$ :
             $a_i^t \sim \pi_\theta(a_i^t | s_i^t)$ ;
        end
        Execute action and store  $(s_t, a_t, r_t, s_{t+1})$ ;
    end
    /* Strategy Update */
    Update  $\pi_\theta$  by minimizing Eq.(41);
    Update  $V_\vartheta$  by minimizing Eq.(43);
end

```

A. Simulation Setup

We implement the proposed U2GNet algorithm using Python 3.7 and PyTorch 1.7.0, and conduct all simulations on a workstation equipped with an Intel Core i9-13900 K workstation. The UAV-UGV collaborative scenario is set within a 1 km x 1 km target area, where several PoIs are randomly distributed. The data volume at each PoI is initialized randomly within the range of [0, 15] Mbit. The simulation environment includes 25 UAVs and 5 UGVs. Each UAV flies at a fixed altitude of 100 m and has a maximum speed of $v_{\max} = 25$ m/s. Additional simulation parameters are summarized in Table I. Each fully connected and GRU layer in U2GNet contains 256 units. The HGAT consists of two graph attention layers, each with four attention heads. The learning rate is set to 1×10^{-4} , the discount factor $\gamma = 0.99$, and the GAE parameter $\lambda = 0.95$. The replay buffer size is 50,000, and the mini-batch size is 128. During the validation phase, we load pre-trained models and select actions based on current state observations to evaluate their performance in unseen test environments. In the testing phase, we reset the environment to assess the generalization capability of the trained models.

B. Baseline Algorithms

To evaluate the performance of our proposed algorithm, we select the following baseline methods that have been applied in similar scenarios for comparison:

- *CommNet*: *CommNet* is a centralized approach that performs communication among all UAVs during training and execution [14], [35].
- *DGRL-Based Method*: This method uses deep graph convolutional reinforcement learning to solve the optimization problem in UAV data collection scenarios [36].
- *DDQN-Based Method*: This is a computation offloading algorithm based on DDQN multi-agent reinforcement learning [18].
- *Random policy*: At each time slot t , each UAV v randomly samples an action from the action space $\mathcal{A}$.

For the non-HGAT-based methods mentioned above, each agent concatenates all locally observed states into a vector, while filling zero values for unobserved entities.

In addition, we also include two graph network methods for comparison. Although they have not been applied in UAV-UGV collaboration scenarios, their architectural features make them suitable as benchmarks for this study:

- *HGN*: This is a novel network structure for multi-robot cooperation under partially observable conditions [37].
- *EHCAMAN*: An entropy-enhanced hierarchical graph continuous action multiagent coordination control method [38].

C. Convergence Analysis

As shown in the Fig. 5, all metrics show consistent improvement over time. For example, in Fig. 5(a), the data collection rate for U2GNet is approximately 82%. In contrast, the performance of HGN and DGRL is noticeably lower, stabilizing at around 70% and 65%, respectively. Additionally, as shown in Fig. 5(b), U2GNet enhances the task completion rate, which converges to approximately 83%. This superior performance is attributed to U2GNet's ability to effectively aggregate observations from both UAVs and UGVs, enabling it to balance their heterogeneity and facilitate efficient task allocation and collaboration.

D. Parameter Sensitivity Analysis

During the simulation process, R_{obs} and R_{comm} are two critical parameters. We use η and ξ as evaluation metrics. The values of R_{obs} and R_{comm} are selected from the sets [30, 60, 90, 120] m and [90, 120, 150, 180, 210, 240] m, respectively. Performance under different parameter combinations is shown in Fig. 6. As observed in Fig. 6(a), increasing R_{obs} and R_{comm} enables UAVs to cover a wider area, thereby improving η . For instance, when $R_{comm} = 180$ m, increasing R_{obs} from 30m to 90 m raises η from 0.72 to 0.84. However, due to limitations in UAV energy and computational resources, excessively large R_{obs} values may increase computational overhead. Furthermore, larger R_{comm} values promote efficient data exchange between nodes and help reduce redundancy in observations. Nevertheless, excessively

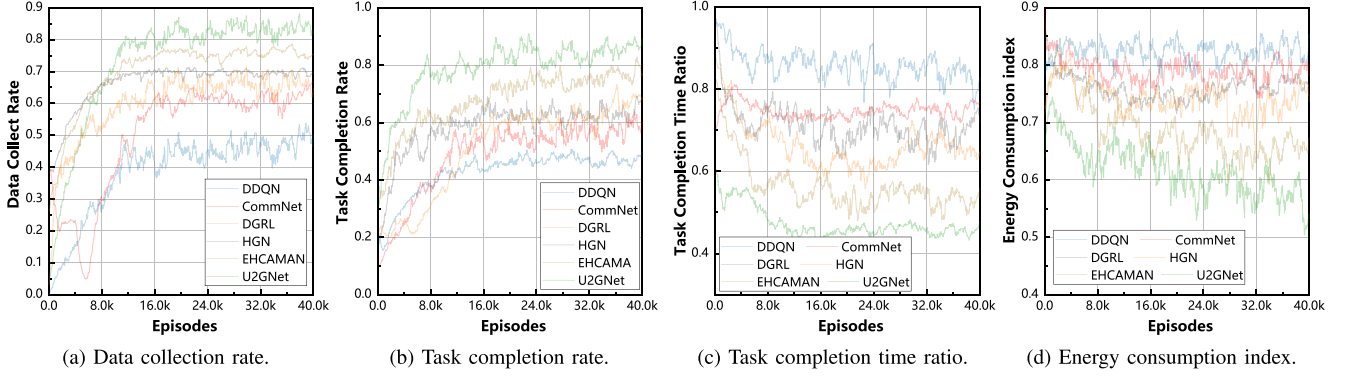


Fig. 5. The learning curves of data collection rate, task completion rate, task completion time ratio, and energy index during the training phase.

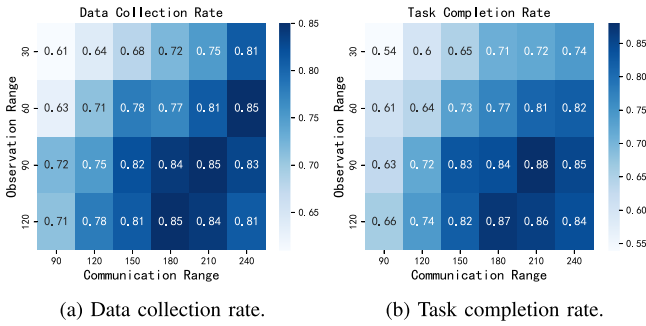


Fig. 6. Impact of R_{obs} and R_{comm} on performance at different values.

increasing R_{comm} may lead to heightened wireless channel interference, which negatively impacts communication quality. Similarly, Fig. 6(b) shows that when R_{obs} and R_{comm} are within the ranges of 60–90 m and 150–180 m, respectively, the task completion rate ξ remains relatively high. To achieve a better balance between performance and cost, we set $R_{obs} = 90$ m and $R_{comm} = 150$ m in our simulation.

E. Ablation Study

To verify the necessity of the HGAT module, we replaced it with a traditional homogeneous GAT(U2GNet(w/GAT)). The experiments are conducted in two different scenarios:

- scenario 1: A small-scale environment with 5 UAVs and 1 UGVs.
- scenario 2: A medium-scale environment with 25 UAVs and 5 UGVs.

As observed in Fig. 7, U2GNet(w/GAT) exhibits performance degradation across all metrics in both scenarios, with this decline being more pronounced in the more complex Scenario 2. Taking task completion rate as an example, U2GNet(w/GAT) demonstrates approximately 20.4% performance reduction in Scenario 2 compared to U2GNet. This indicates that the homogeneous GAT fails to effectively distinguish different node types and their semantic differences during message passing, resulting in inadequate characterization of cross-type collaborative relationships. In contrast, HGAT constructs more expressive feature

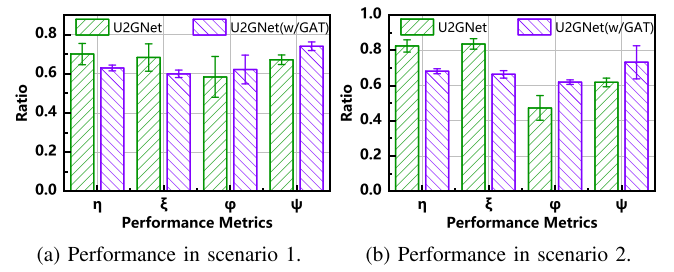


Fig. 7. Performance comparison of using GAT and HGAT modules in different scenarios.

TABLE II
MEAN AND STANDARD DEVIATION OF FIVE METRICS UNDER DIFFERENT SCENARIOS

Method	Scenario	Times (ms)	η	ξ	ϕ	ψ
U2GNet	scenario 1	11.22	0.701 ± 0.004	0.684 ± 0.081	0.584 ± 0.103	0.672 ± 0.044
	scenario 2	20.18	0.825 ± 0.034	0.836 ± 0.030	0.473 ± 0.070	0.618 ± 0.024
U2GNet (w/o H1)	scenario 1	10.01	0.650 ± 0.045	0.621 ± 0.059	0.602 ± 0.053	0.711 ± 0.032
	scenario 2	17.54	0.749 ± 0.014	0.724 ± 0.011	0.579 ± 0.205	0.682 ± 0.034

representations based on node types and interaction patterns, thereby demonstrating superior performance advantages in heterogeneous collaborative tasks. We conducted a comparative analysis between the complete U2GNet model and its variant with the first-layer HGAT removed, denoted as U2GNet(w/o H1). As shown in Table II, in scenario 1, U2GNet achieves approximately 9.2% improvement in task completion rate compared to U2GNet(w/o H1). In scenario 2, this improvement margin further expands to approximately 19.02%. Furthermore, U2GNet(w/o H1) causes UAVs to perform ineffective maneuvers when observing nearby UGVs, thereby increasing energy consumption. As observed in Table II, removing the first-layer HGAT reduces the computation time for single-step decision making. This computational efficiency gain primarily stems from the omission of observation aggregation operations, where

TABLE III
THE MEAN AND STANDARD DEVIATION OF FIVE METRICS ($U=25, V=5$)

Method	η	ξ	ϕ	ψ	Times (ms)
U2GNet	0.825 ± 0.034	0.836 ± 0.030	0.473 ± 0.070	0.618 ± 0.024	20.18
U2GNet (w/o C)	0.805 ± 0.025	0.813 ± 0.015	0.601 ± 0.042	0.672 ± 0.051	19.63
U2GNet (w/o G)	0.741 ± 0.021	0.747 ± 0.042	0.561 ± 0.080	0.656 ± 0.045	19.10

agents bypass the matrix multiply-accumulate computations of the multi-head attention mechanism and directly process simplified input features.

Subsequently, we designed two variants to isolate the contributions of inter-agent communication and GRU.

- *U2GNet(w/o C)*: Disable the inter-agent communication mechanism (i.e., neither sending nor receiving messages), rendering the second-layer HGAT ineffective, and only utilizes the first-layer HGAT to process self-observations along with GRU.
- *U2GNet(w/o G)*: Remove the GRU module and replace it with a simple feedforward layer.

Table III shows that U2GNet(w/o C) with disabled communication experiences approximately 1.46% reduction in task completion rate, with other performance metrics declining accordingly. This occurs because after communication is disabled, UAVs can only rely on their own observations and historical memory for decision making, without access to neighboring state information. U2GNet(w/o C) achieves a single-step decision time of 19.63 ms, slightly lower than U2GNet. This is attributed to the second-tier HGAT no longer performing message passing and aggregation operations, thereby saving computational overhead. Furthermore, as shown in Table III, U2GNet(w/o G) without GRU exhibits approximately 10.65% reduction in task completion rate, highlighting the critical role of GRU in modeling temporal dependencies and supporting sequential decision making. U2GNet achieves a per-round decision time of 20.18 ms, slightly higher than its variants due to the inherent computational overhead introduced by the attention mechanism in HGAT and recurrent operations in GRU. Nevertheless, this time cost remains within an acceptable range.

F. Communication Dropout Rate Analysis

We train two variants: *U2GNet(trained with dropout)* and *U2GNet(trained without dropout)*. The former is trained under communication dropout conditions (with a dropout probability $p = 0.5$), while the latter is trained without dropout. During testing, both models are evaluated under varying communication dropout rates, where $p \in [0, 1]$ with a step size of 0.1. As shown in Fig. 8, as the communication dropout rate increases, the task completion time for *U2GNet(trained without dropout)* continuously increases. In contrast, *U2GNet(trained with dropout)* maintains relatively stable task completion times even when the communication dropout rate exceeds 0.4. Additionally, as shown on the second vertical axis of Fig. 8, when p increases from 0 to

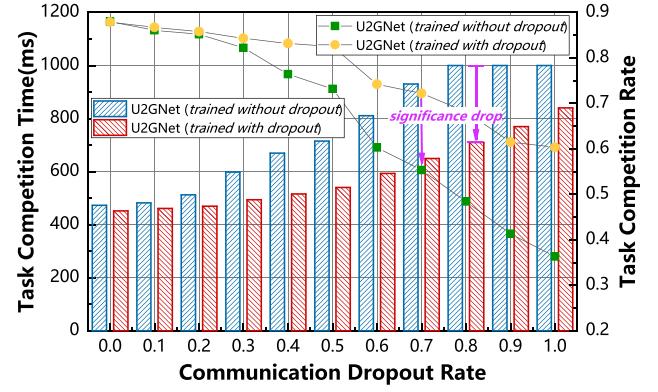


Fig. 8. Impact of communication failure on U2GNet.

TABLE IV
ENVIRONMENTAL PARAMETERS FOR DIFFERENT INTERFERENCE SCENARIOS

Parameter	Suburban	Urban	Dense Urban	Disaster
a	4.88	9.61	11.95	27.23
b	0.43	0.16	0.136	0.08
η_L (dB)	1.0	1.0	1.6	2.3
η_N (dB)	20.0	23.0	28.0	34.0

TABLE V
TASK COMPLETION RATE UNDER DIFFERENT SCENARIOS

Method	Suburban	Urban	Dense Urban	Disaster
U2GNet (Ours)	0.94	0.82	0.71	0.58
DDQN	0.78	0.61	0.46	0.31
DGRL	0.85	0.68	0.53	0.39
HGN	0.89	0.74	0.61	0.47

1.0, the task completion rate of the model trained with dropout decreases by only 0.28, while the model trained without dropout experiences a decrease of 0.51. The above results indicate that *U2GNet(trained with dropout)* experiences a smaller performance degradation in unreliable communication environments. This verifies that introducing a communication dropout simulation mechanism during training improves U2GNet's adaptability in actual deployments.

To evaluate the robustness of U2GNet in extreme environments with frequent communication link disruptions, such as natural disasters, we conducted experiments across three disruption scenarios by adjusting the LoS probability parameters. The probability parameters are shown in Table IV.

1) *Task Completion Rate*: As shown in Table V, as the propagation environment progressively deteriorates from suburban to disaster scenarios (with decreasing LoS probability), the offloading rates of all methods decrease due to increased frequency of link disruptions. In the suburban, U2GNet achieves a task completion rate of 0.94, outperforming DDQN, DGRL, and HGN by 20.51%, 10.59%, and 5.62%, respectively. In the disaster, it achieves a task completion rate of 0.58, surpassing DDQN's 0.31, as U2GNet can allocate attention weights to reliable line-of-sight links when communication quality degrades. HGN employs a graph attention mechanism and achieves a task completion rate of 0.47, but due to the absence of an

TABLE VI
TASK COMPLETION TIME RATIO UNDER DIFFERENT SCENARIOS

Method	Suburban	Urban	Dense Urban	Disaster
U2GNet (Ours)	0.38	0.47	0.59	0.73
DDQN	0.56	0.71	0.84	0.92
DGRL	0.48	0.62	0.75	0.87
HGN	0.42	0.54	0.66	0.79

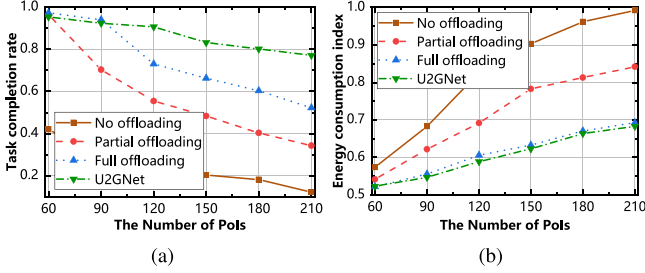


Fig. 9. Performance comparison of different offloading schemes with different number of PoIs.

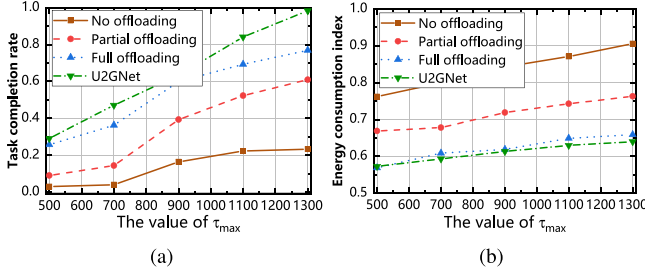


Fig. 10. Performance comparison of different offloading schemes under different $\tau_{\max}$ values.

inter-group aggregation layer, its performance is 18.97% lower than U2GNet.

2) *Task Completion Time Ratio*: As shown in Table VI, as communication disruptions intensify, other baseline methods approach the time slot limit due to repeated transmission failures and suboptimal offloading decisions. In contrast, U2GNet achieves a task completion time ratio of 0.73 even in disaster scenarios, 20.65% lower than DDQN. This is because U2GNet prioritizes strong LoS links by optimizing offloading ratios and allocating computational resources, thereby avoiding time losses from repeated retransmissions.

G. Performance Analysis

To evaluate the performance of U2GNet, we compare it with three benchmark offloading schemes: 1) *No offloading*: all data are processed locally on the UAV; 2) *Full offloading*: all data are offloaded to the UGV for processing; 3) *Partial offloading*: data are evenly split between local computation and UGV processing.

1) *Impact of Different Numbers of PoIs on Different Offloading Schemes*: Fig. 9 demonstrates that U2GNet consistently achieves the highest offloading ratio and lowest energy consumption compared to all benchmark strategies. This superior performance stems from its dynamic adjustment of offloading

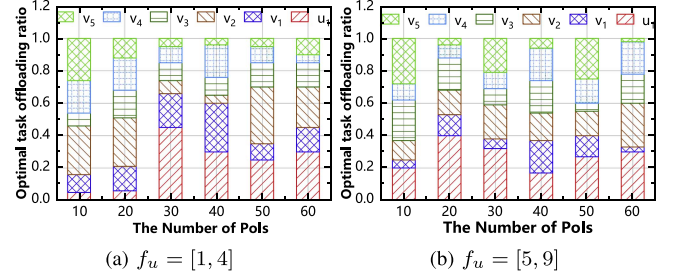


Fig. 11. Optimization of offloading ratio with different computing resources.

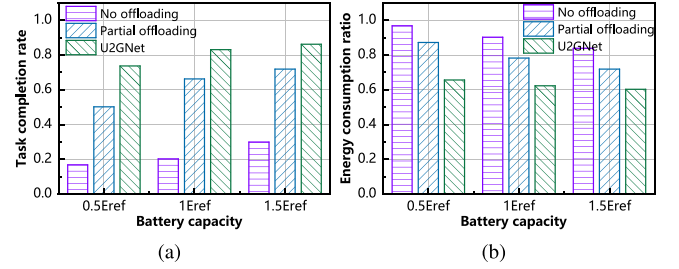


Fig. 12. Performance comparison of different offloading methods under different battery capacities.

ratios based on UAV remaining computational resources and proximity to UGVs. In contrast, the *No Offloading* strategy yields the lowest offloading ratio by relying entirely on onboard processing, while *Full Offloading* suffers performance degradation due to computational bottlenecks at UGVs. For energy consumption, U2GNet maintains a relatively stable growth trend, whereas *No Offloading* exhibits the steepest rise in UAV energy usage, highlighting its inefficiency under heavy workloads. The increasing energy cost of *Full Offloading* indicates that communication overhead dominates its energy profile. Although the *Partial offloading* scheme provides a trade-off between performance and efficiency, its performance is still significantly lower than U2GNet.

2) *Impact of Different $\tau_{\max}$ Values on Different Offloading Schemes*: Furthermore, the performance of U2GNet may be influenced by variations in the latency constraint. We keep the data acquisition time unchanged and increase $\tau_{\max}$ from 500 ms to 1300 ms. At this time, T_{proc} is in the range of 200 ms to 1000 ms. In Fig. 10(a), the task completion rate improves with increasing $\tau_{\max}$. When $\tau_{\max} = 500$ ms, the shorter processing time (200 ms) results in a lower completion rate. As can be seen from Fig. 10(b), the UAV's energy consumption increases slightly. This is because the $e_{u,clt}^{(t)}$ remains unchanged due to the fixed collection time. Moreover, the UAV's own $C_{u,m}$ (CPU cycles/bit) is fixed parameter, which have little impact on local and offloading energy consumption. However, as $\tau_{\max}$ increases, the volume of data $D_u^{(t)}$ that can be processed also grows, resulting in longer processing time, which will also cause a moderate rise in overall energy consumption.

3) *Optimal Offloading Ratio Under Different Numbers of PoIs*: In Fig. 11, we evaluate the optimal offloading ratio between UAVs and UGVs under varying numbers of PoIs, ranging

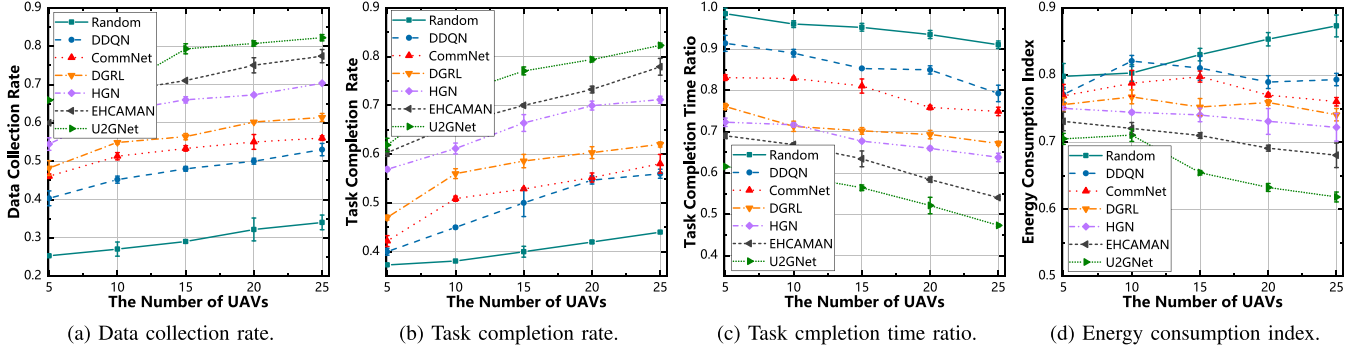


Fig. 13. Impact of different number of UAVs on performance. U2GNet consistently outperforms baselines across all metrics.

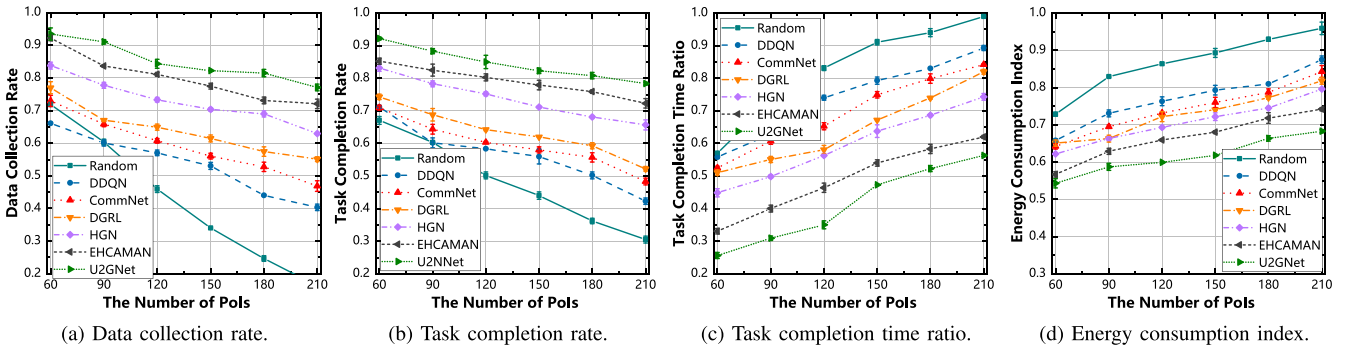


Fig. 14. Impact of different number of PoIs on performance. U2GNet shows smallest performance degradation compared to baselines.

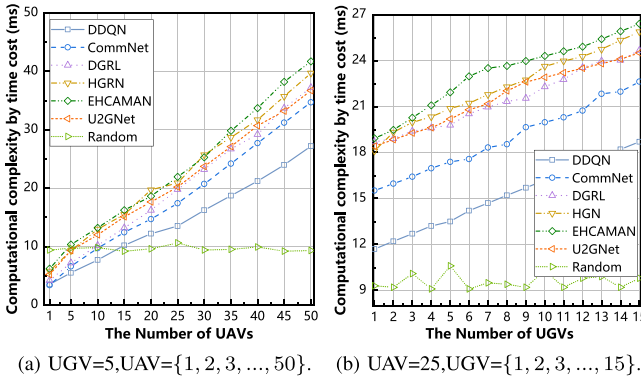


Fig. 15. Impacts of number of UAVs and UGVs on computational complexity by time cost (ms).

from 10 to 60. The data volume for each PoI is randomly generated within the range of $[0, 15]$ Mbit. As shown in Fig. 11(a), when the number of PoIs is 10 or 20, the majority of data is offloaded to UGVs. This is because the overall task load is relatively low, and the UGVs possess stronger computational capabilities, resulting in lower energy and latency costs. As the number of PoIs increases, the system experiences a rapid surge in data volume, prompting more collaborative processing between UAVs and UGVs. In addition, we evaluate the task allocation performance of U2GNet when f_u is $[5, 9]$ GHz. As depicted in Fig. 11(b), the proportion of tasks processed locally by the UAV

increases. With a higher f_u , the UAV achieves greater computational efficiency, thereby reducing its reliance on UGVs. This enables the system to flexibly distribute tasks between UAVs and UGVs based on the availability of computational resources.

As shown in Fig. 12, when the battery capacity is low, U2GNet's task completion rate is approximately 0.73, which is higher than that of other offloading methods. In terms of energy efficiency, U2GNet achieves a value of 0.657, representing a 24.74% reduction compared to the *Partial Offloading* method. This improvement is attributed to U2GNet's dynamic adjustment of the offloading ratio and computational resources, which prioritizes offloading data to the UGV to maximize UAV energy savings. In contrast, the *Partial Offloading* scheme employs a fixed offloading strategy, which can lead to an imbalanced energy distribution and is more prone to transmission interruptions under low battery conditions. Moreover, as the battery capacity increases, the marginal benefit of the dynamic optimization strategy gradually diminishes when resources are abundant, reducing U2GNet's performance advantage, though it still outperforms other offloading schemes.

4) *Impact of UAV and PoI Density on System Performance:* We evaluate performance under varying system scales by adjusting the number of UAVs (from 5 to 25) and the number of PoIs (from 60 to 200), while keeping other parameters fixed. For each configuration, we conduct 10 independent runs and record the average results along with the corresponding standard. As shown in Fig. 13, U2GNet consistently outperforms the six

baseline methods across all performance metrics. For example, in Figs. 13(a) and (b), both the data collection rate and task completion rate increase with the number of UAVs under a fixed number of PoIs. This is intuitive, as more UAVs can cover larger areas and collect data from more PoIs. Taking UAV=25 as an example, U2GNet's task completion rate is 32.73% higher than DGRL. This improvement is attributed to DGRL's use of GCN to model spatial dependencies, where the convolution kernels are fixed, preventing dynamic adjustment of the influence from different neighbors. Conversely, when the number of UAVs is fixed, Fig. 14 shows that the performance of all methods declines as the number of PoIs increases. This occurs because a higher number of PoIs implies larger data volumes, creating greater difficulties in improving data collection rates. In such cases, UAVs must expend more energy and time to complete tasks. Compared to other baseline algorithms, U2GNet's task completion rate is relatively insensitive to changes in the number of PoIs. Furthermore, the memory module enables UAVs to leverage historical information, thereby sustaining higher task completion rates under challenging conditions.

H. Discussion and Scalability Analysis

To assess U2GNet's scalability, we evaluate its computational complexity across environments of different scales. As shown in Fig. 15(a), the computational complexity of all DRL-based algorithms increases with the number of UAVs. This occurs because the time cost of action generation depends on the number of entities M in the environment, which determines the computational complexity of the actor network during the forward process (see Section IV-C). In contrast, the *Random* policy incurs minimal computational overhead since, in each time slot t , the action for each UAV is randomly sampled from the action space. *CommNet* exhibits higher computational costs due to its centralized communication architecture, which requires all-to-all agent communication and results in quadratic complexity growth, limiting its scalability. U2GNet's computational cost is slightly higher owing to message passing in observation and communication layers, as well as inter-group feature aggregation. However, U2GNet requires only about 20.18 ms per decision cycle to compute actions for 25 UAVs, while achieving the highest task completion rate (see Fig. 13). Similarly, Fig. 15(b) indicates that computational complexity increases with the number of UGVs for all DRL methods. However, U2GNet adopts distributed information passing and inter-group feature aggregation mechanisms, allowing UAVs to communicate with only a subset of UGVs, reducing the global observation burden. When the number of UGVs reaches 15, U2GNet consumes only about 24.5 ms, maintaining relatively low computational overhead.

Furthermore, in mixed-agent environments that include fixed IoT devices such as sensors, U2GNet's HGAT framework can be extended to additional entity groups with limited added complexity, as static nodes like sensors may reduce movement-related computations and leverage range-limited interactions. In future work, we plan to further investigate this extension to better ensure scalable performance in diverse IoT scenarios.

VI. CONCLUSION

This paper presents U2GNet, a novel framework for UAV-UGV in smart city environments. By integrating heterogeneous graph attention network with deep reinforcement learning, the U2GNet effectively addresses the challenges of heterogeneous agent collaboration. Experimental results demonstrate that U2GNet significantly outperforms baseline methods in performance metrics, including data collection rates, task completion rate and energy efficiency.

REFERENCES

- [1] A. Wasicek, "The future of 5G smart home network security is micro-segmentation," *Netw. Secur.*, vol. 2020, no. 11, pp. 11–13, 2020.
- [2] C. Dutriez, O. S. Oubbati, C. Gueguen, and A. Rachedi, "Energy efficiency relaying election mechanism for 5G Internet of Things: A deep reinforcement learning technique," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2024, pp. 1–6.
- [3] Q. Wu, Q. Liu, W. Zhu, and Z. Wu, "Energy efficient UAV-assisted IoT data collection: A graph-based deep reinforcement learning approach," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 6, pp. 6082–6094, Dec. 2024.
- [4] Y. Liu, Q. Deng, Z. Zeng, A. Liu, and Z. Li, "A hybrid optimization framework for age of information minimization in UAV-Assisted MCS," *IEEE Trans. Serv. Comput.*, vol. 18, no. 2, pp. 527–542, Mar./Apr. 2025.
- [5] Z. Dai, C. H. Liu, R. Han, G. Wang, K. K. Leung, and J. Tang, "Delay-sensitive energy-efficient UAV crowdsensing by deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2038–2052, Apr. 2023.
- [6] H. V. Abeywickrama, Y. He, E. Dutkiewicz, and B. A. Jayawickrama, "An adaptive UAV network for increased user coverage and spectral efficiency," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2019, pp. 1–6.
- [7] Y. Gao, J. Tao, Y. Xu, Z. Wang, Y. Gao, and M. Wang, "Improving user QoE via joint trajectory and resource optimization in multi-UAV assisted MEC," *IEEE Trans. Serv. Comput.*, vol. 18, no. 3, pp. 1472–1486, May/Jun. 2025.
- [8] F. Pervez, A. Sultana, C. Yang, and L. Zhao, "Energy and latency efficient joint communication and computation optimization in a multi-UAV-assisted MEC network," *IEEE Trans. Wireless Commun.*, vol. 23, no. 3, pp. 1728–1741, Mar. 2024.
- [9] Y. Chen, Y. Yang, Y. Wu, J. Huang, and L. Zhao, "Joint trajectory optimization and resource allocation in UAV-MEC systems: A Lyapunov-assisted DRL approach," *IEEE Trans. Serv. Comput.*, vol. 18, no. 2, pp. 854–867, Mar./Apr. 2025.
- [10] Q. Wu, Q. Liu, Y. He, and Z. Wu, "UAV-enabled energy-efficient aerial computing: A federated deep reinforcement learning approach," *IEEE Trans. Rel.*, vol. 74, no. 3, pp. 3559–3572, Sep. 2025.
- [11] Y. Wang et al., "Task offloading for post-disaster rescue in unmanned aerial vehicles networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1525–1539, Aug. 2022.
- [12] J. Park, S. Choi, T. Kim, C. Lee, and S. Lee, "Public bus-assisted task offloading for UAVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 12, pp. 20561–20573, Dec. 2024.
- [13] M. Dai, Z. Su, Q. Xu, and N. Zhang, "Vehicle assisted computing offloading for unmanned aerial vehicles in smart city," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 3, pp. 1932–1944, Mar. 2021.
- [14] S. Jung, C. Park, M. Levorato, J.-H. Kim, and J. Kim, "Two-stage self-adaptive task outsourcing decision making for edge-assisted multi-UAV networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 11, pp. 14889–14905, Nov. 2023.
- [15] Q. Tang, B. Li, H. H. Yang, Y. Li, S. He, and K. Yang, "Delay and load fairness optimization with queuing model in multi-UAV assisted MEC: A deep reinforcement learning approach," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 2, pp. 1247–1258, Apr. 2025.
- [16] Y. Chen, J. Zhao, Y. Wu, J. Huang, and X. S. Shen, "Multi-user task offloading in UAV-assisted LEO satellite edge computing: A game-theoretic approach," *IEEE Trans. Mobile Comput.*, vol. 24, no. 1, pp. 363–378, Jan. 2025.
- [17] H. Huang, Z.-Y. Chai, B.-S. Sun, H.-S. Kang, and Y.-J. Zhao, "Multi-objective deep reinforcement learning for computation offloading and trajectory control in UAV-base station assisted MEC," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31805–31821, Oct. 2024.

- [18] T. Lyu, Y. Xu, F. Liu, H. Xu, and Z. Han, "Task offloading and resource allocation for satellite-terrestrial integrated networks," *IEEE Internet Things J.*, vol. 12, no. 1, pp. 262–275, Jan. 2025.
- [19] N. Sharma and K. Kumar, "A novel latency-aware resource allocation and offloading strategy with improved prioritization and DDQN for edge-enabled UDNs," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 6, pp. 6260–6272, Dec. 2024.
- [20] S. Wang, X. Song, T. Song, and Y. Yang, "Fairness-aware computation offloading with trajectory optimization and phase-shift design in RIS-assisted multi-UAV MEC network," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 20547–20561, Jun. 2024.
- [21] S. Shen, G. Shen, Z. Dai, K. Zhang, X. Kong, and J. Li, "Asynchronous federated deep-reinforcement-learning-based dependency task offloading for UAV-assisted vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31561–31574, Oct. 2024.
- [22] H. Xu, Y. Shao, L. Liu, P. Shan, J. Guo, and W. Xu, "Deadline-aware and energy-efficient distributed scheduling algorithm in ground-air cooperative networks," in *Proc. 9th Int. Conf. Comput. Commun. Syst.*, 2024, pp. 789–795.
- [23] P. Chen, L. Luo, D. Guo, X. Luo, X. Li, and Y. Sun, "Secure task offloading for rural area surveillance based on UAV-UGV collaborations," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 923–937, Jan. 2024.
- [24] J. Yan, X. Zhao, and Z. Li, "Deep-reinforcement-learning-based computation offloading in UAV-assisted vehicular edge computing networks," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19882–19897, Jun. 2024.
- [25] Y. Zhao, C. H. Liu, T. Yi, G. Li, and D. Wu, "Energy-efficient ground-air-space vehicular crowdsensing by hierarchical multi-agent deep reinforcement learning with diffusion models," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 12, pp. 3566–3580, Dec. 2024.
- [26] D. Zhang, Z. Zhang, J. Zhang, T. Zhang, L. Zhang, and H. Chen, "UAV-assisted task offloading system using dung Beetle optimization algorithm & deep reinforcement learning," *Ad Hoc Netw.*, vol. 156, 2024, Art. no. 103434.
- [27] P. Qin, Y. Fu, R. Ding, and H. He, "Competition-awareness partial task offloading and UAV deployment for multitier parallel computational Internet of Vehicles," *IEEE Syst. J.*, vol. 18, no. 3, pp. 1753–1764, Sep. 2024.
- [28] H. Guo, Y. Wang, J. Liu, and C. Liu, "Multi-UAV cooperative task offloading and resource allocation in 5G advanced and beyond," *IEEE Trans. Wireless Commun.*, vol. 23, no. 1, pp. 347–359, Jan. 2024.
- [29] J. Alotaibi, O. S. Oubbati, M. Atiquzzaman, F. Alromithy, and M. R. Altmanian, "Optimizing disaster response with UAV-mounted RIS and hap-enabled edge computing in 6G networks," *J. Netw. Comput. Appl.*, vol. 241, 2025, Art. no. 104213.
- [30] Y. Yuan, P. Zhang, Z. Wang, and Y. Chen, "Noncooperative event-triggered control strategy design with round-robin protocol: Applications to load frequency control of circuit systems," *IEEE Trans. Ind. Electron.*, vol. 67, no. 3, pp. 2155–2166, Mar. 2020.
- [31] L. Lin, X. Liao, H. Jin, and P. Li, "Computation offloading toward edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1584–1607, Aug. 2019.
- [32] O. S. Oubbati, J. Alotaibi, F. Alromithy, M. Atiquzzaman, and M. R. Altmanian, "A UAV-UGV cooperative system: Patrolling and energy management for urban monitoring," *IEEE Trans. Veh. Technol.*, vol. 74, no. 9, pp. 13521–13536, Sep. 2025.
- [33] C. S. D. Witt et al., "Is independent learning all you need in the starcraft multi-agent challenge?," 2020, *arXiv:2011.09533*.
- [34] W. Liu et al., "Joint trajectory design and resource allocation in UAV-enabled heterogeneous MEC systems," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 30817–30832, Oct. 2024.
- [35] W. J. Yun et al., "Cooperative multiagent deep reinforcement learning for reliable surveillance via autonomous multi-UAV control," *IEEE Trans. Inform.*, vol. 18, no. 10, pp. 7086–7096, Oct. 2022.
- [36] Z. Ye, K. Wang, Y. Chen, X. Jiang, and G. Song, "Multi-UAV navigation for partially observable communication coverage by graph reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4056–4069, Jul. 2023.
- [37] Y. Tang, J. He, and Z. Zhao, "Activity-aware human mobility prediction with hierarchical graph attention recurrent network," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 2, pp. 1604–1616, Feb. 2025.
- [38] Y. Chen, K. Wang, G. Song, and X. Jiang, "Entropy enhanced multiagent coordination based on hierarchical graph learning for continuous action space," *IEEE Trans. Cogn. Devel. Syst.*, vol. 16, no. 3, pp. 1161–1171, Jun. 2024.



Qianqian Wu received the BS degree in computer technology from the North China University of Science and Technology, Tangshan, China, in 2022. She is currently working toward the PhD degree with the School of Computer Science and Technology, Beijing Jiaotong University. Her current research interests include UAV aided networks, mobile crowdsensing, and deep reinforcement learning.



Qiang Liu (Member, IEEE) received the BS and PhD degrees in communication and information system from the Beijing Institute of Technology, Beijing, China, in 2002 and 2007, respectively. From 2007 to 2018, he was an assistant professor with the School of Computer Science and Technology, Beijing Jiaotong University, where he has been an associate professor, since 2019. He is the author of more than 50 articles and four patents. His research interests include mobile ad-hoc networks, UAV networking, wireless communications, wireless routing, and swarm intelligence.



Ying He (Senior Member, IEEE) received the BEng degree in telecommunications engineering from the Beijing University of Posts and Telecommunications, Beijing, China, in 2009, and the PhD degree in telecommunications engineering from the University of Technology Sydney, Australia, in 2017. She is currently a senior lecturer with the School of Electrical and Data Engineering, University of Technology Sydney. Her research interests include physical layer algorithms in wireless communication with machine learning, vehicular communication, spectrum sharing, and satellite communication.



Zefan Wu received the BS degree in mathematics and applied mathematics and in computer science and technology in 2023 from Beijing Jiaotong University, where he is currently working toward the PhD degree with the School of Computer Science and Technology. His current research interests include deep reinforcement learning, mobile edge computing, and UAV communications.